

spca Extended Vignette

Giovanni Maria Merola

2026-10-03

Package spca

*This vignette shows how to use the **spca** package to compute least squares sparse principal component analysis (LS-SPCA). LS-SPCA replaces the principal components of a dataset with sparse components that maximise the explained variance and stay closely aligned with the principal components. The vignette is an instructional, condensed version of the submitted *Journal of Statistical Software* paper: it concentrates on the package and its workflow, illustrated with real datasets. Theory and proofs are in the references.*

Keywords: SPCA, LS-SPCA, variance explained, projection, uncorrelated, R.

Introduction

The **spca** package computes least squares sparse principal component analysis (LS-SPCA). LS-SPCA replaces the principal components (PCs) of a dataset with *sparse* components (sPCs): linear combinations of a few variables that maximise the explained variance while remaining closely aligned with the PCs. Unlike conventional SPCA, it yields (nearly) uncorrelated sPCs that genuinely maximise the variance explained.

This vignette is a condensed, instructional version of the submitted *Journal of Statistical Software* paper on the package. It focuses on using **spca**; for the theory, derivations, and proofs see Merola (2015) and Merola and Chen (2019).

The package provides a fast *C++* backend, with separate engines for tall ($n > p$) and fat ($n < p$) matrices, and an *R* interface to fit, inspect, and compare solutions. Users choose among computational variants, forward, stepwise, or backward variable selection, stopping rules, and target approximation levels. Helper functions and the `print()`, `summary()`, and `plot()` methods evaluate fits numerically and visually, including against standard PCA, and let solutions computed by other packages be assessed in the same framework.

LS-SPCA in brief

Let X be the $n \times p$ centred data matrix and $t_j = Xa_j$ the j th sPC, with sparse weights a_j . LS-SPCA computes the weights by minimising the least-squares data approximation

$$\min_{a_j} \|X - t_j b_j^\top\|^2, \quad \text{subject to} \quad \text{card}(a_j) < p, \quad a_j^\top S a_i = 0, \quad i < j; \quad j = 1, \dots, r,$$

that is, subject to a cardinality limit and to uncorrelatedness with the previously computed sPCs.

The package implements three variants:

- *uSPCA* — the optimal, exactly uncorrelated solution.
- *cSPCA* — uses residual (deflated) matrices in place of the uncorrelatedness constraints; the sPCs are mildly correlated and often of lower cardinality.
- *pSPCA* — a regression-based projection; the fastest variant, used for variable selection.

Variables are selected by forward, stepwise, or backward search, stopping when a target cumulative variance explained (CVEXP) or R^2 with the PCs is reached. Full details are in Merola (2015) and Merola and Chen (2019).

The `spca` package

The `spca` package fits and compares sparse PCA solutions through a single interface. It uses two backends, one for tall matrices ($n > p$) and one for fat matrices ($n < p$), selected automatically from the input. The main fitting function, `spca()`, accepts either a data matrix or a covariance matrix: a square symmetric matrix is treated as a covariance matrix, otherwise as a data matrix. Character arguments are matched on their first letter. The fat backend requires the data matrix; PC scores are computed only when a data matrix is supplied.

Fitted models are returned as objects of class `spca`, holding both weights and percentage contributions, the nonzero indices and cardinality of each sPC, the VEXP and CVEXP and their proportions relative to the PCs, the sPC correlation matrix, and the squared correlations `r2` with the PCs. Scores are stored when computed. The package adds helper functions and `print()`, `summary()`, and `plot()` methods. Table-producing methods can return the underlying tables; plotting functions can return *ggplot2* objects for further editing.

Computations run in the *C++* backend, which uses referenced arrays to avoid deep copies. Fat matrices are handled with a reverse-SVD in the row space. The covariance matrix $S = X^\top X$ and the product SS are formed once, and deflations use rank-1 updates rather than recomputation, reducing the per-step cost from about $O(p^3)$ to $O(p^2)$. The leading eigen-pair can optionally be computed with the power method (`pm_weights = TRUE` for weights, `pm_varsel = TRUE` for selection), controlled by the `maxiter_pm...` and `eps_pm...` arguments.

A typical SPCA workflow using `spca`

To illustrate a standard SPCA workflow, we use the classic Holzinger–Swineford dataset, available from the **psychTools** package (Revelle 2025). To keep the results compact, we use a subset of $n = 145$ observations and $p = 12$ ability-test results, treated as variables in the analysis. The same subset is used in other analyses (for example, Ferrara et al. 2019, and references therein). The variables are centred and scaled to unit variance. The dataset is included in the package as `holzinger`; the accompanying factor `holzinger_scales` identifies which of the four scales each variable belongs to.

When evaluating and comparing LS-SPCA solutions, we emphasize percentage contributions, which are weights scaled to unit ℓ_1 norm (the sum of their absolute values), rather than conventional weights scaled to unit ℓ_2 norm. Contributions are generally easier to interpret, and the **spca** methods print and plot them by default. Set `contributions = FALSE` to obtain conventional weights.

The aim is to obtain a compact representation of the relationships among the 12 variables while retaining most of the variation represented by ordinary PCA. We first use PCA to choose a candidate number of components. We then examine how sparsification affects variance explained, component interpretation, and agreement with the original PCs. Finally, we compare alternative solutions to assess whether additional sparsity or restrictions based on the test scales are appropriate.

We assess how closely SPCA solutions approximate PCA using relative variance explained (RVEXP) and relative cumulative variance explained (RCVEXP), defined as the VEXP and CVEXP of the sPCs divided by the corresponding quantities for the PCs. We also consider the correlation between each sPC and its corresponding PC.

Load the data as follows.

```
R> data("holzinger")
R> dim(holzinger)
```

```
[1] 145 12
```

```
R> data("holzinger_scales")
```

PCA

Although it is not required before fitting sparse components, examining the PCA solution helps identify the components to be approximated and choose how many to retain.

The `pca()` function performs an eigendecomposition of the covariance matrix and stores the result in an object of classes `pca` and `spca`. It wraps a specialized C++ routine that uses a standard eigendecomposition for tall matrices and a truncated inverse singular value decomposition for fat matrices. Its arguments are:

```
pca(M, n_comps = NULL, center_data = FALSE, scale_data = FALSE,  
    fat_matrix = NULL, screeplot = TRUE, qq_plot = FALSE,  
    n_obs = NULL, neigen_toplot = NULL, cor = TRUE,  
    common_var = NULL, pm = FALSE, eps_pm = 1e-04,  
    maxiter_pm = 1000)
```

`M` can be either a data matrix or a covariance matrix. PC scores are not computed when a covariance matrix is supplied. The default `n_comps = NULL` computes all possible components. If `pm = TRUE`, only `n_comps` PCs are computed with the power method, controlled by the convergence tolerance `eps_pm` and maximum number of iterations `maxiter_pm`. Because the returned object also inherits from `spca`, the methods for that class apply to it.

`pca()` can generate a conventional screeplot and a Wachter qq-plot (Wachter 1976). The latter compares the observed eigenvalues with quantiles of the Marchenko–Pastur distribution, the limiting eigenvalue distribution for random covariance matrices of independent variables with a common variance (Winkler 2021). It therefore applies to covariance matrices whose variables have the same variance, specified through `common_var`, and to correlation matrices when `cor = TRUE`. For covariance or correlation matrix input, `n_obs` must be supplied to `pca()` or later to `mp_qqplot()` unless it is already stored in the fit. The methods `screeplot()` and `mp_qqplot()` provide further customization.

The following plots show the screeplot and qq-plot, with a line fitted to all but the three largest eigenvalues.

```
R> ho_pca = pca(holzinger, screeplot = TRUE, qq_plot = FALSE)
```

```
R> mp_qqplot(ho_pca, n_fitline = -4)
```

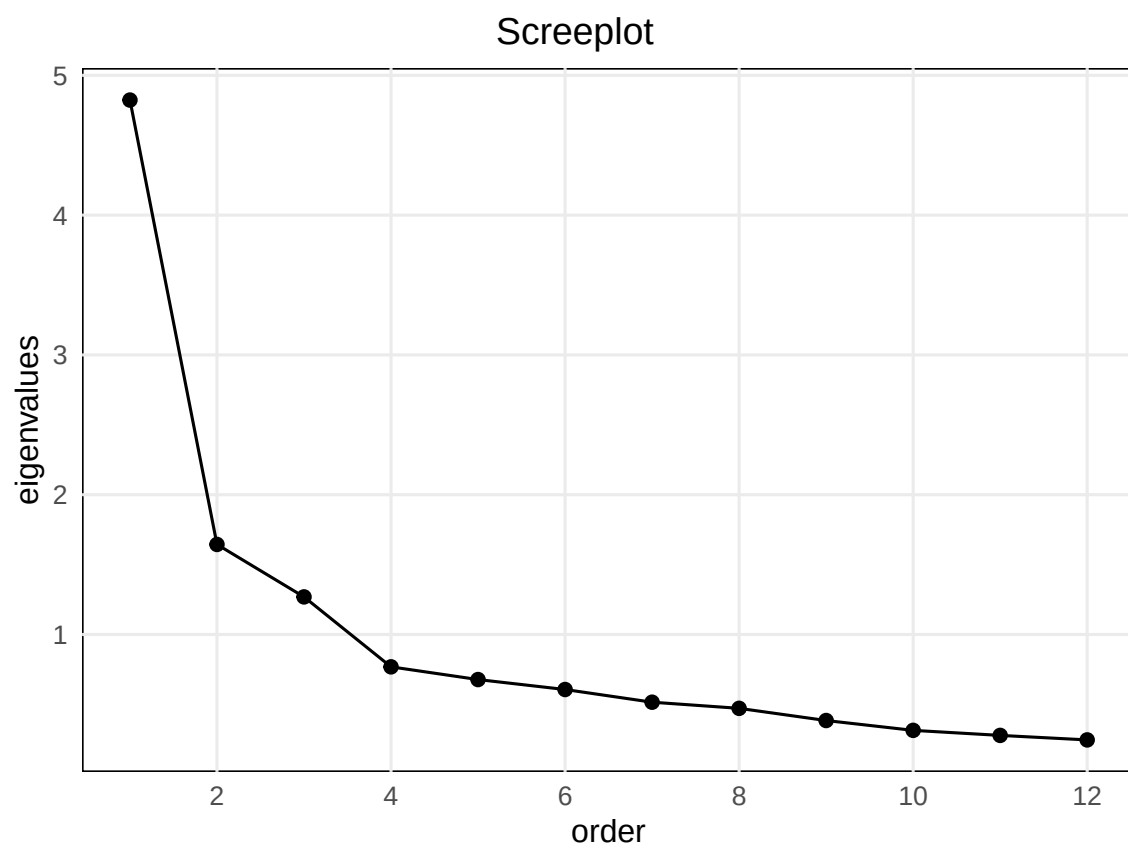
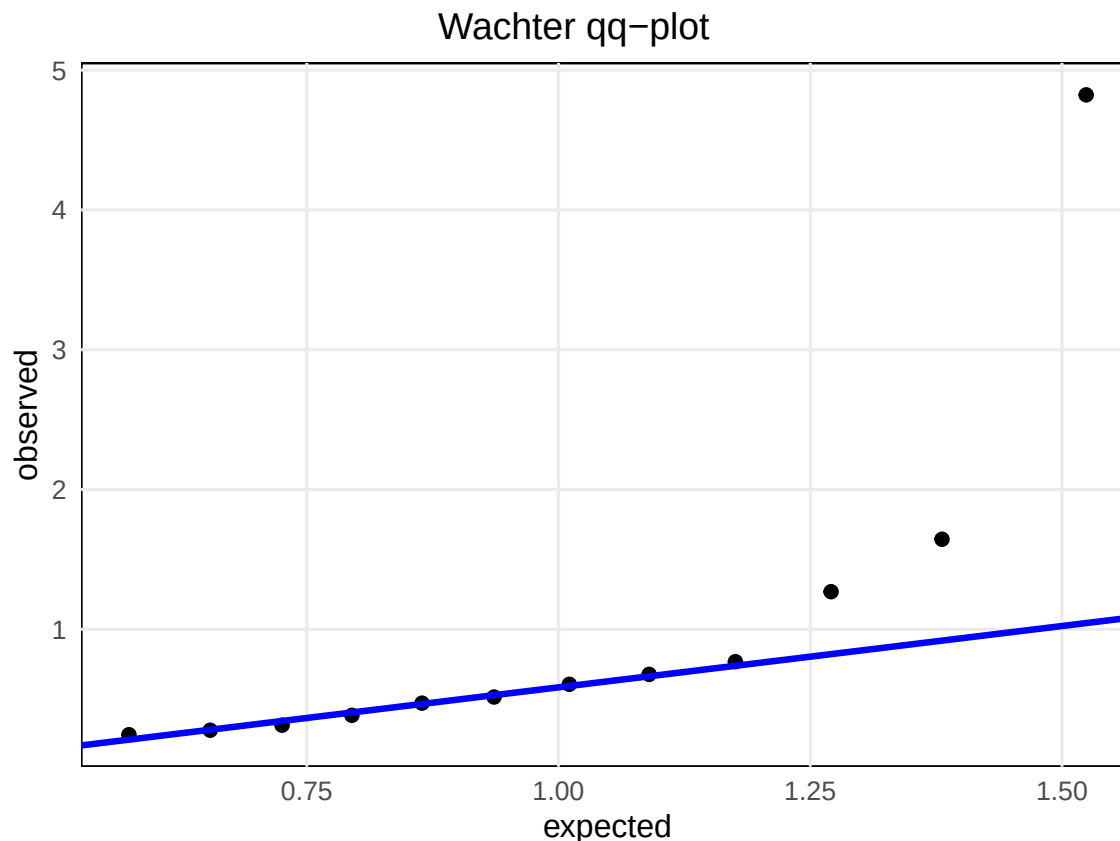


Figure 1: Screeplot.



The screeplot suggests retaining four components, whereas the qq-plot indicates that only three eigenvalues exceed those expected under a random correlation matrix. We nonetheless retain four components. Since components are extracted sequentially, the fourth does not affect the first three, and it illustrates how LS-SPCA behaves on a weak-signal component.

The following bar plots show the variable contributions to the first four PCs, coloured by scale. The first PC is a weighted average of all variables, with similar weights. Disregarding small contributions, the second PC mainly contrasts verbal (VBL) and speed (SPD) abilities, with an additional contrast between two mathematics (MTH) tests. The third contrasts spatial (SPL) abilities with verbal and speed abilities. The fourth has no clearly interpretable pattern, possibly because it primarily captures noise.

```
R> plot(ho_pca, n_plot = 4, variable_groups = holzinger_scales)
```

SPCA

The main function, `spca()`, invokes the C++ engines that compute LS-SPCA solutions and stores the result in an `spca` object. Its arguments are:

```
spca(M, alpha = 0.95, n_comps = NULL, ncomp_by_cvexp = NULL,
     method = c("cspca", "uspca", "pspca"),
     var_selection = c("fwd", "bkward", "step"),
     objective = c("cvexp", "r2"), intensive = FALSE,
     fat_matrix = NULL, fixed_index_list = list(),
     center_data = FALSE, scale_data = FALSE,
     pm_weights = FALSE, eps_pm_weights = 1e-05,
     maxiter_pm_weights = 100, pm_varsel = FALSE,
     eps_pm_varsel = 1e-05, maxiter_pm_varsel = 200)
```

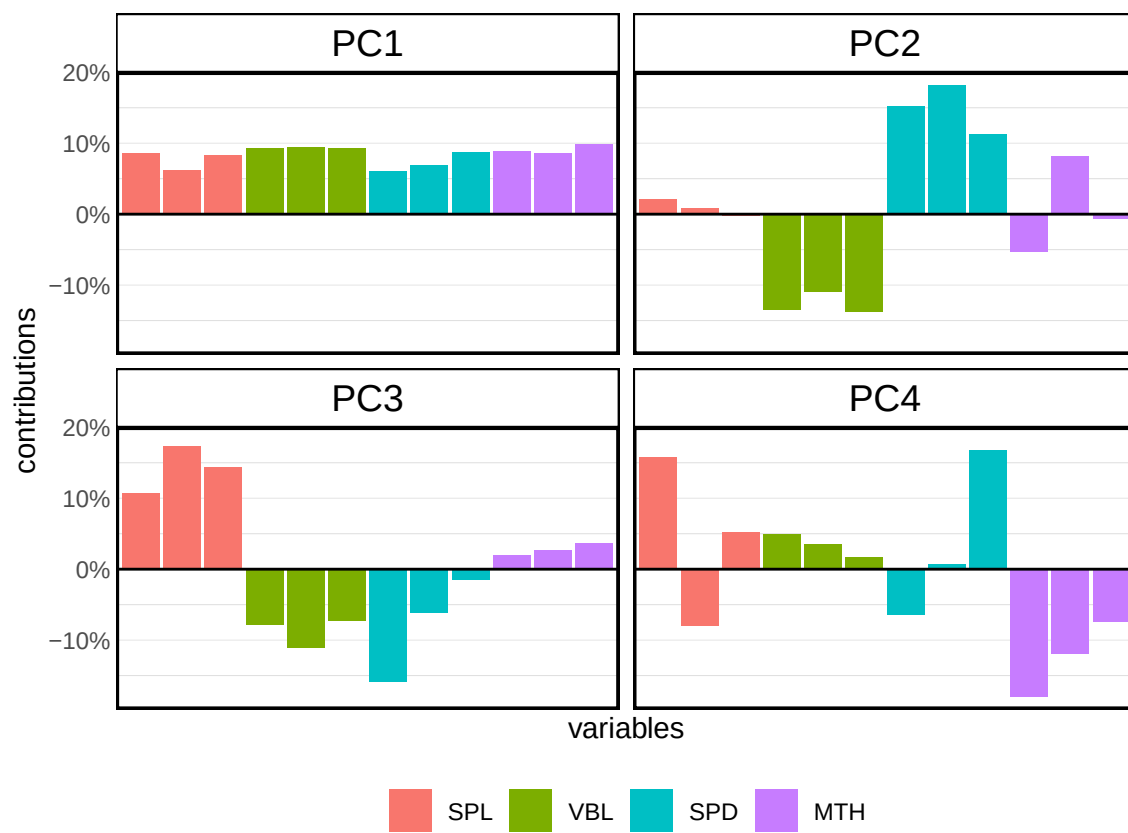


Figure 2: Contributions to the first four PCs.

Character arguments may be abbreviated: only the first character of the first supplied value is used.

We fit four components using the defaults. The number of components can instead be left unspecified and computation stopped when `ncomp_by_cvexp` is reached. If both arguments are supplied, `n_comps` takes priority.

```
R> ho_spcadef = spca(
+   M = holzinger,
+   n_comps = 4,           # four components
+   alpha = 0.95,         # 95% CVEXP
+   method = "c",         # cSPCA
+   var_selection = "f",  # forward selection
+   objective = "cvexp",  # stop by CVEXP
+   intensive = FALSE     # select by R-squared
+ )
```

The final four arguments are written explicitly for clarity. The same fit is obtained with `ho_spcadef = spca(holzinger, 4)`.

`summary()`

The `summary()` method reports metrics for assessing an `spca` object relative to the PCs. With the default `cor_with_pc = TRUE`, it includes the correlations between the sPCs and their corresponding PCs. These correlations are omitted when the model was fitted from a covariance matrix.

- **Vexp**: percentage variance explained.
- **Cvexp**: percentage cumulative variance explained.
- **Rvexp**: variance explained relative to the corresponding PC.
- **Rcvexp**: cumulative variance explained relative to the corresponding PCs.
- **Card**: number of nonzero weights.
- **r**: correlation between each sPC and its corresponding PC.

Summary of the SPCA fit.

```
R> summary(ho_spcadef)
```

	sPC1	sPC2	sPC3	sPC4
Vexp	38.6%	12.9%	9.9%	6.1%
Cvexp	38.6%	51.5%	61.4%	67.5%
Rvexp	96.0%	94.5%	93.5%	95.3%
Rcvexp	96.0%	95.6%	95.3%	95.3%
Card	7	4	4	4
r	0.978	0.946	0.925	0.762

The weight cardinalities are substantially smaller than the 12 available variables, while all sPCs attain more than 95% RCVEXP. The first three sPCs are strongly correlated with their respective PCs. The fourth correlation is weaker, plausibly because that component contains less signal, as suggested by the qq-plot. The correlation matrix of the sPCs follows.

```
R> show_correlations(ho_spcadef)
```

	sPC1	sPC2	sPC3	sPC4
sPC1	1.00	-0.01	-0.03	-0.02
sPC2	-0.01	1.00	-0.08	-0.10
sPC3	-0.03	-0.08	1.00	-0.06
sPC4	-0.02	-0.10	-0.06	1.00

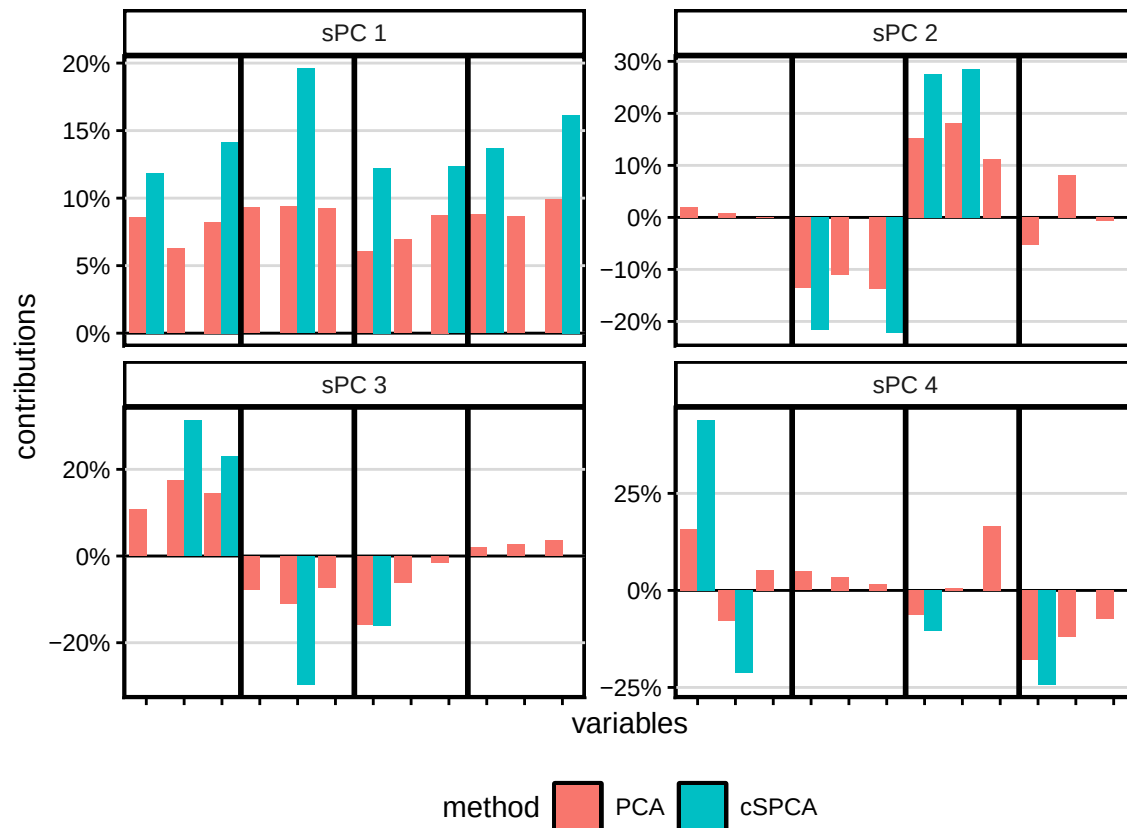
sPC-PC	0.98	0.95	0.92	0.76

Although cSPCA permits correlated sPCs, their mutual correlations are small in this example. The larger correlations involving the fourth sPC provide further evidence that it conveys comparatively little signal.

Comparison of SPCA with PCA

This comparison is not required in a standard SPCA analysis, but it clarifies the reduction achieved relative to PCA. `compare_spca()` produces tables and plots for two or more solutions. Here it displays the PCA and SPCA contributions side by side and plots them by scale.

```
R> compare_spca(
+   list(ho_pca, ho_spca),
+   variable_groups = holzinger_scales,
+   print_weights = TRUE,
+   print_summary = TRUE,
+   methods_names = c("PCA", "cSPCA")
+ )
```



Percentage contributions

	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
visual	8.6	11.9	2.0		10.7		15.8	43.9
cubes	6.2		0.9		17.3	31.4	-7.9	-21.3
flags	8.2	14.2	-0.1		14.4	23.0	5.3	
paragraph	9.3		-13.5	-21.6	-7.8		4.9	
sentence	9.4	19.6	-11.0		-11.0	-29.7	3.5	
wordm	9.3		-13.8	-22.3	-7.2		1.6	
addition	6.0	12.2	15.2	27.5	-15.9	-15.9	-6.4	-10.6
counting	6.9		18.2	28.6	-6.1		0.6	

straight	8.7	12.3	11.2	-1.4	16.7	
deduct	8.8	13.7	-5.2	1.9	-17.9	-24.3
numeric	8.6		8.1	2.6	-11.9	
series	9.9	16.1	-0.6	3.7	-7.4	

Summary statistics

	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
Vexp	40.2%	38.6%	13.7%	12.9%	10.6%	9.9%	6.4%	6.1%
Cvexp	40.2%	38.6%	53.9%	51.5%	64.5%	61.4%	70.9%	67.5%
Rvexp	100.0%	96.0%	100.0%	94.5%	100.0%	93.5%	100.0%	95.3%
Rcvexp	100.0%	96.0%	100.0%	95.6%	100.0%	95.3%	100.0%	95.3%
Card	12	7	12	4	12	4	12	4
abs_r	1.00	0.98	1.00	0.95	1.00	0.92	1.00	0.76

The cSPCA components preserve the main patterns in the PCs while involving fewer variables. The first sPC remains a positive weighted average spanning all four scales. The second more clearly contrasts verbal and speed abilities after eliminating the mathematics contributions. The third retains the contrast between spatial abilities and verbal and speed abilities. The fourth remains difficult to interpret at the scale level despite its sparser representation, because variables within the same scales receive contributions of opposite signs. Variable selection in LS-SPCA is not equivalent to removing the variables with the smallest absolute PCA weights: for example, the first sPC retains **addition** despite its small PCA weight.

`aggregate_by_group()` sums contributions within each scale. These totals can aid interpretation at the scale level, although contributions with opposite signs within a scale cancel.

cSPCA contributions aggregated by scale.

```
R> aggregate_by_group(
+   ho_spcadef,
+   variable_groups = holzinger_scales,
+   only_nonzero = FALSE
+ )
```

Percentage contributions

	sPC1	sPC2	sPC3	sPC4
SPL	26.0%		54.4%	22.6%
VBL	19.6%	-43.9%	-29.7%	
SPD	24.6%	56.1%	-15.9%	-10.6%
MTH	29.8%			-24.3%

Analysis of the sparse solutions

`print()`

`print()` is the default method and accepts the following arguments:

```
print.spca(x, cols = NULL, only_nonzero = TRUE,
           contributions = TRUE, digits = 3,
           thresh_card = 1e-07, return_table = FALSE,
           component_names = NULL, ...)
```

It displays percentage contributions by default. With `only_nonzero = TRUE`, variables that do not load on any sPC are omitted. `cols` selects the columns to print; an integer selects the first `cols` columns.

Nonzero contributions to the cSPCA components.

```
R> ho_spcadef
```

Percentage contributions

	sPC1	sPC2	sPC3	sPC4
visual	11.9%			43.9%
cubes			31.4%	-21.3%
flags	14.2%		23.0%	
paragraph		-21.6%		
sentence	19.6%		-29.7%	
wordm		-22.3%		
addition	12.2%	27.5%	-15.9%	-10.6%
counting		28.6%		
straight	12.3%			
deduct	13.7%			-24.3%
series	16.1%			
	-----	-----	-----	-----
Cvexp	38.6%	51.5%	61.4%	67.5%

plot()

The `plot()` method can be customized through the following arguments. Entries under `controls` are **ggplot2** parameters included for convenience. Set `return_plot = TRUE` to save and modify the resulting plot.

```
plot.sPCA(  
  x,  
  n_plot = NULL,  
  plot_type = c("bars", "circular", "heatmap"),  
  contributions = TRUE,  
  only_nonzero = TRUE,  
  pc_weights = NULL,  
  variable_groups = NULL,  
  plot_title = NULL,  
  return_plot = FALSE,  
  show_plot = TRUE,  
  controls = list(  
    color_scale = c("ggplot", "cbb", "printsafe", "bw"),  
    variable_names = c("none", "auto"),  
    legend_position = c("none", "bottom", "right", "top", "left"),  
    grid_type = c("horizontal", "full", "none"),  
    facet_labels = NULL,  
    legend_title = NULL,  
    x_axis_lab = "variables",  
    adjust_labels_circ = NULL,  
    flip_heatmap = FALSE,  
    heatmap_color_range = c("values", "unit")  
  ),  
  ...  
)
```

Supplying PC weights through `pc_weights` adds them to the plots for comparison, except for circular plots. Supplying a vector or factor through `variable_groups` identifies groups of variables, as in the PCA plot above. The default is a conventional bar plot.

```
R> plot(ho_spcadev, controls = list(variable_names = "auto"))
```

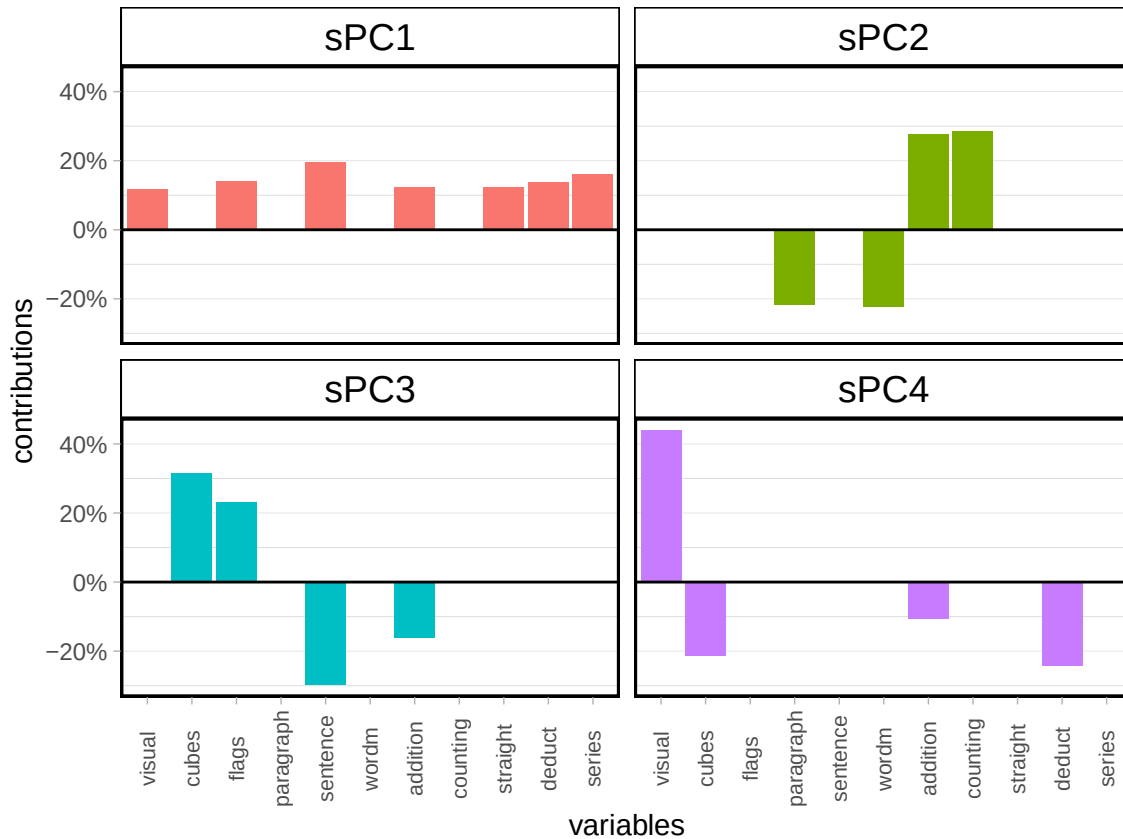


Figure 3: Contributions to each sPC.

Set `plot_type = "c"` for a more compact circular plot. Here `color_scale = "printsafe"` keeps the tones distinguishable when printed in grayscale.

```
R> plot(
+   ho_spcadev,
+   n_plot = 3,
+   plot_type = "c",
+   controls = list(
+     color_scale = "printsafe",
+     variable_names = "auto"
+   )
+ )
```

The third option is a heat map. Adding the PC contributions permits a visual comparison. The default `heatmap_color_range = "values"` uses the observed range, which is more informative here than fixing the colour scale between -1 and 1.

```
R> plot(
+   ho_spcadev,
+   pc_weights = ho_pca$contributions,
+   plot_type = "h",
+   controls = list(variable_names = "none")
+ )
```

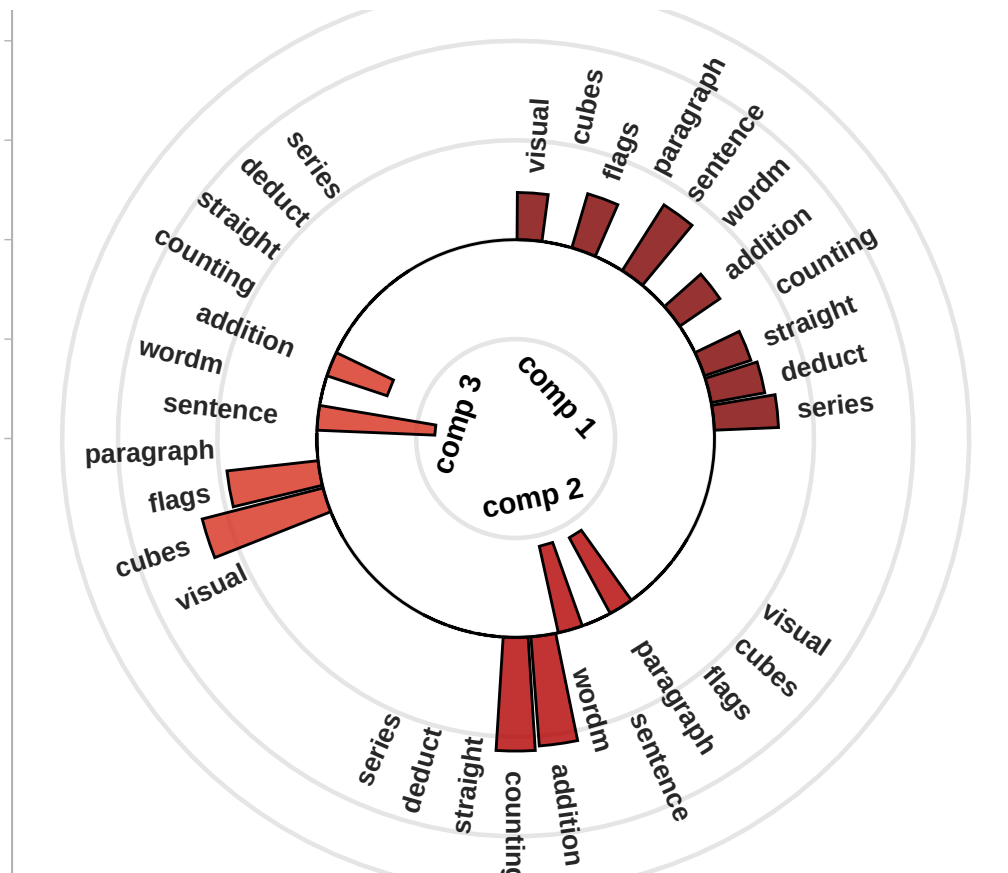


Figure 4: Circular plots of the contributions to the first three sPCs.

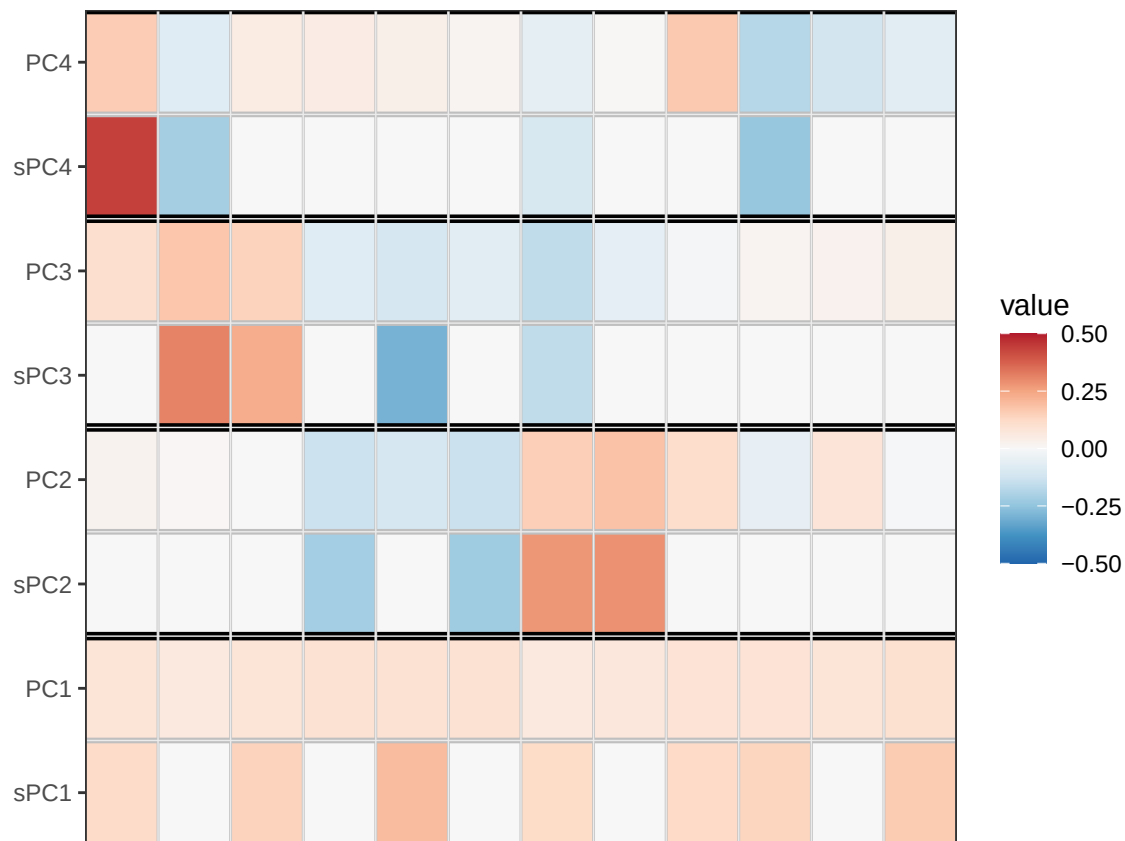


Figure 5: Heat maps comparing cSPCA and PCA contributions.

Comparison of two LS-SPCA solutions

We now compare the default cSPCA fit with a fit that uses the forward intensive selection procedure. Both use the default $\alpha = 0.95$.

```
R> ho_cspcai = spca(
+   holzinger,
+   n_comps = 4,
+   alpha = 0.95,
+   method = "c",
+   objective = "cvexp",
+   intensive = TRUE
+ )
```

The method names are used in the comparative plot legend. `col_short_names = TRUE` keeps the printed tables compact, while `color_scale = "cbb"` selects a colour-blind-friendly palette.

Comparative summaries for default cSPCA (M1) and cSPCA with intensive selection (M2).

```
R> compare_spca(
+   list(ho_spcadef, ho_cspcai),
+   plot_weights = FALSE,
+   variable_groups = holzinger_scales,
```

```
+ print_weights = FALSE,
+ print_summary = TRUE,
+ col_short_names = TRUE,
+ methods_names = c("cSPCA", "Intensive")
+ )
```

Summary statistics

	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
Vexp	38.6%	38.7%	12.9%	13.0%	9.9%	9.8%	6.1%	6.0%
Cvexp	38.6%	38.7%	51.5%	51.7%	61.4%	61.5%	67.5%	67.5%
Rvexp	96.0%	96.2%	94.5%	95.2%	93.5%	92.7%	95.3%	93.1%
Rcvexp	96.0%	96.2%	95.6%	95.9%	95.3%	95.4%	95.3%	95.2%
Card	7	7	4	4	4	4	4	3
abs_r	0.98	0.98	0.95	0.93	0.92	0.92	0.76	0.77

Contributions for default cSPCA (M1) and cSPCA with intensive selection (M2).

```
R> compare_spc(
+ list(ho_spcadef, ho_cspcai),
+ plot_weights = FALSE,
+ variable_groups = holzinger_scales,
+ print_weights = TRUE,
+ print_summary = FALSE,
+ col_short_names = TRUE,
+ methods_names = c("cSPCA", "Intensive")
+ )
```

Percentage contributions

	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
visual	11.9						43.9	50.1
cubes					31.4	31.2	-21.3	-17.1
flags	14.2	12.3			23.0	23.6		
paragraph		14.9	-21.6	-19.9				
sentence	19.6	13.7			-29.7	-31.2		
wordm			-22.3	-18.1				
addition	12.2		27.5	29.9	-15.9	-14.0	-10.6	
counting			28.6	32.2				
straight	12.3	16.9						
deduct	13.7	12.4					-24.3	-32.8
numeric		12.6						
series	16.1	17.1						

Both fits meet the 95% RCVEXP target for every component. With the same cardinalities for the first three sPCs (7, 4, and 4), intensive selection (M2) achieves slightly higher RCVEXP than default cSPCA (M1). For the fourth sPC, M2 selects three rather than four variables, with a modest decrease in RCVEXP from 95.3% to 95.2%. Intensive selection therefore gives a slightly closer approximation for the first three components and a sparser fourth component while still meeting the target.

The contributions and plot show that the methods share five of the seven variables selected for the first sPC, despite selecting partially different sets. They select the same variables for the second and third sPCs, with slightly different contributions. For the fourth sPC, M2 retains a subset of the variables selected by M1, omitting **addition**. Assessing the relative interpretability of these weight sets requires subject-matter expertise.

```

R> compare_spca(
+   list(ho_spcadef, ho_cspcai),
+   plot_weights = TRUE,
+   variable_groups = holzinger_scales,
+   col_grouplines = "firebrick2",
+   color_scale = "cbb",
+   print_weights = FALSE,
+   print_summary = FALSE,
+   col_short_names = TRUE,
+   methods_names = c("cSPCA", "Intensive")
+ )

```

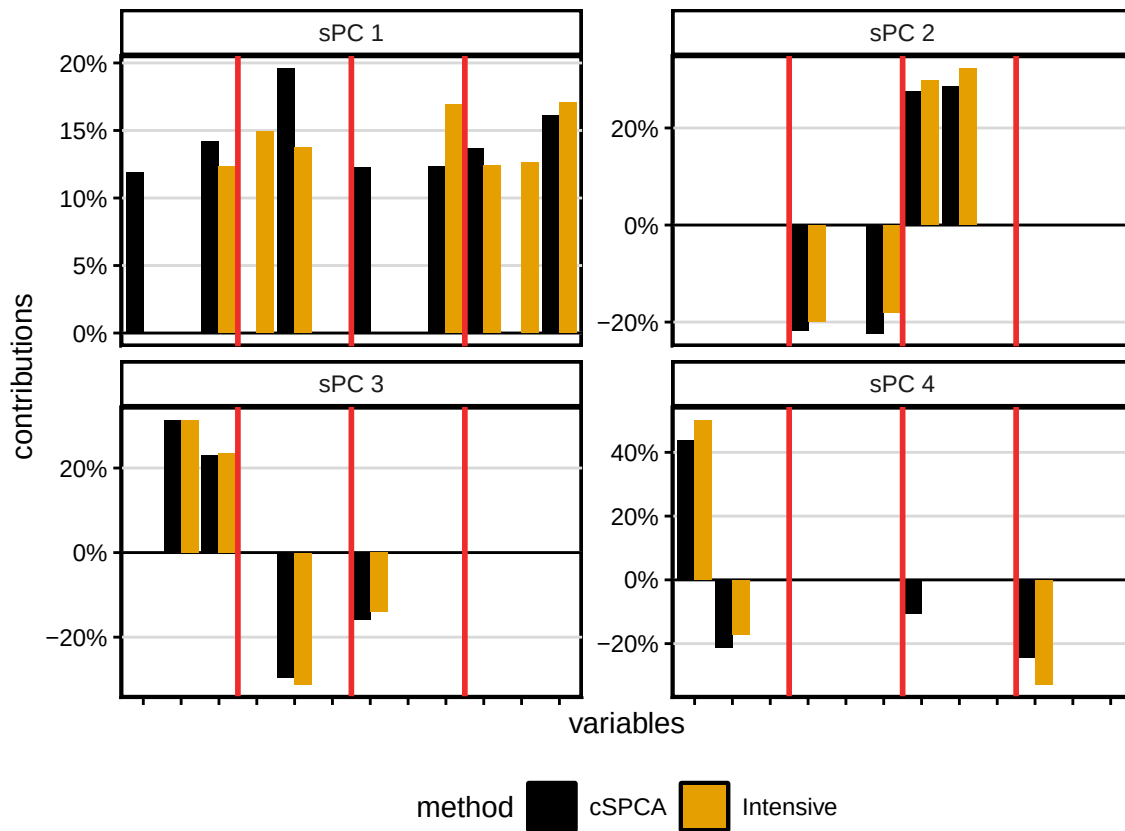


Figure 6: Contributions from default cSPCA and cSPCA with intensive selection.

Fixed indices

The `fixed_index_list` argument allows a subset of variables to be specified for each sPC weight set. This is useful when the nonzero indices obtained by another method are available.

To illustrate, we obtain four components in which each sPC loads only on the variables belonging to the corresponding scale. Constraining the weights in this way is not generally recommended because LS-SPCA aims to explain variation in the entire dataset rather than within separate subsets.

```

R> ho_spcafixed = spca(
+   holzinger,
+   alpha = 0.95,

```

```
+   n_comps = 4,
+   fixed_index_list = holzinger_scales
+ )
```

Contributions when each sPC loads on one scale.

```
R> ho_spcafixed
```

Percentage contributions

	sPC1	sPC2	sPC3	sPC4
visual	41.7%			
cubes	22.2%			
flags	36.0%			
paragraph		24.5%		
sentence		44.5%		
wordm		31.0%		
addition			52.3%	
counting			41.8%	
straight			5.9%	
deduct				-22.4%
numeric				-41.5%
series				36.1%
	-----	-----	-----	-----
Cvexp	27.2%	46.3%	61.2%	66.1%

Summary of the fits on distinct scales.

```
R> summary(ho_spcafixed, cor_with_pc = TRUE)
```

	sPC1	sPC2	sPC3	sPC4
Vexp	27.2%	19.1%	14.9%	5.0%
Cvexp	27.2%	46.3%	61.2%	66.1%
Rvexp	67.6%	139.6%	141.0%	77.4%
Rcvexp	67.6%	85.9%	94.9%	93.3%
Card	3	3	3	3
r	0.761	-0.497	-0.414	0.344

These constrained approximations are parsimonious and easy to interpret, but they do not closely approximate the PCA solution. The second and third sPCs are negatively correlated with their PCs. This has no numerical significance because eigenvectors are defined only up to sign, but it can make comparisons confusing. `change_sign()` reverses selected components and updates all affected quantities. The following call then displays only the correlations with the corresponding PCs.

```
R> ho_spcafixed = change_sign(ho_spcafixed, index_to_change = 2:3)
R> show_correlations(ho_spcafixed, type = "pcs")
```

	PC1	PC2	PC3	PC4
sPC-PC	0.76	0.5	0.41	0.34

Create an spca object

`new_spca()` converts a set of weights, including weights computed by another SPCA method, into an `spca` object to which the package methods can be applied. It requires the weights and either a covariance matrix or the original data matrix.

```
R> A = cbind(ho_spcadef$weights[, 1], ho_cspcai$weights[, 2])
R> ho_spcahyb = new_sPCA(A, X = holzinger, method_name = "hybrid")

R> is.sPCA(ho_spcahyb)
```

```
[1] TRUE
```

Practical recommendations

The default settings are a practical starting point for a wide range of datasets. The default cSPCA method provides an intermediate choice between the exactly uncorrelated uSPCA components and the computationally simpler pSPCA solution. Unlike uSPCA, cSPCA does not impose a minimum weight cardinality and avoids uncorrelatedness constraints that can cause numerical difficulties for later components. pSPCA may offer a modest speed advantage over cSPCA, but it can yield more strongly correlated components and, in some cases, higher cardinalities.

Forward selection is recommended for routine use. Stepwise selection and intensive search increase computational cost and are most appropriate for small to medium matrices. Intensive search can bring RCVEXP closer to the target and often produces more parsimonious solutions. Backward elimination is particularly expensive and should be used cautiously, mainly with small matrices. The default `objective = "cvexp"` directly controls cumulative variance explained relative to PCA. The alternative `objective = "r2"` may reduce computation time, although it controls a different approximation criterion.

The power method can accelerate computation, particularly for larger matrices. Convergence may be slow when the two leading eigenvalues of the relevant eigenproblem are close, which can occur for later components. Users should check convergence warnings and revert to the default eigensolver when necessary. As a general rule, the power method is most useful for very large matrices.

The following table summarizes these recommendations. Matrix size refers primarily to the number of variables p for the tall-matrix backend, although n also affects the initial covariance computation. Both n and p matter for the fat-matrix backend. The categories are qualitative because cost also depends on the number of components, their cardinalities, and the available hardware.

Setting	Small	Medium	Large
Stepwise selection	Consider	Consider	Avoid
Intensive selection	Consider	Use cautiously	Avoid
Backward elimination	Use cautiously	Avoid	Avoid
<code>objective = "cvexp"</code>	Consider	Consider	Use cautiously
Power method	Little expected gain	Little expected gain	Consider

These recommendations concern computational cost and do not guarantee an improved solution.

Application to larger datasets

We briefly apply LS-SPCA to three larger datasets to demonstrate how the method scales. The same workflow used for `holzinger` applies; the fitted objects below are loaded from the vignette's saved-results file so that the computationally intensive fits are not rerun during installation.

MSSCQ data

The Multidimensional Sexual Self-Concept Questionnaire dataset (MSSCQ; Open-Source Psychometrics Project 2019) contains 16,985 responses to 100 items after observations with more than three zero values are excluded. The variables were centred and standardized to unit variance.

We computed four cSPCA components with $\alpha = 0.95$ and $\alpha = 0.90$.

```
R> mss_cspca95 = spca(mss, n_comps = 4)
R> mss_cspca90 = spca(mss, n_comps = 4, alpha = 0.90)
```

Summary statistics for the MSSCQ fits with $\alpha = 0.95$ and $\alpha = 0.90$.

```
R> compare_spca(
+   list(mss_cspca95, mss_cspca90),
+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
+   plot_weights = FALSE
+ )
```

Summary statistics

	C1.95%	C1.90%	C2.95%	C2.90%	C3.95%	C3.90%	C4.95%	C4.90%
Vexp	23.2%	22.1%	9.4%	8.8%	6.7%	6.4%	3.2%	3.0%
Cvexp	23.2%	22.1%	32.7%	31.0%	39.4%	37.4%	42.6%	40.4%
Rvexp	95.2%	90.7%	94.7%	88.8%	94.9%	90.4%	94.9%	89.1%
Rcvexp	95.2%	90.7%	95.0%	90.2%	95.0%	90.2%	95.0%	90.1%
Card	12	6	17	8	15	9	25	10
abs_r	0.97	0.95	0.97	0.93	0.96	0.93	0.95	0.68

At $\alpha = 0.95$, the four sPCs have cardinalities 12, 17, 15, and 25, with RCVEXP of at least 95%. At $\alpha = 0.90$, their cardinalities fall to 6, 8, 9, and 10 while RCVEXP remains above 90%.

```
R> compare_spca(
+   list(mss_cspca95, mss_cspca90),
+   x_axis_var_names = FALSE,
+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
+   print_tables = FALSE
+ )
```

The resulting sPCs are virtually uncorrelated.

```
R> mss_cors
```

	95%SPC1	95%SPC2	95%SPC3	95%SPC4	90%SPC1	90%SPC2	90%SPC3	90%SPC4
sPC1	1.00	0.00	0.01	-0.01	1.00	-0.01	0.00	0.00
sPC2	0.00	1.00	0.01	0.01	-0.01	1.00	0.02	0.03
sPC3	0.01	0.01	1.00	0.01	0.00	0.02	1.00	0.08
sPC4	-0.01	0.01	0.01	1.00	0.00	0.03	0.08	1.00

Isolet data

The Isolet dataset (Cole and Fanty 1991) contains 618 spectral coefficients and phonetic measures from speakers pronouncing the 26 letters of the alphabet, for a total of 7,797 observations. The variables were standardized to unit variance before analysis.

We computed four cSPCA components with $\alpha = 0.95$ and $\alpha = 0.90$.

```
R> is_cspca95 = spca(iss, n_comps = 4)
R> is_cspca90 = spca(iss, n_comps = 4, alpha = 0.90)
```

Summary statistics for the Isolet fits with $\alpha = 0.95$ and $\alpha = 0.90$.

```
R> compare_spca(
+   list(is_cspca95, is_cspca90),
```

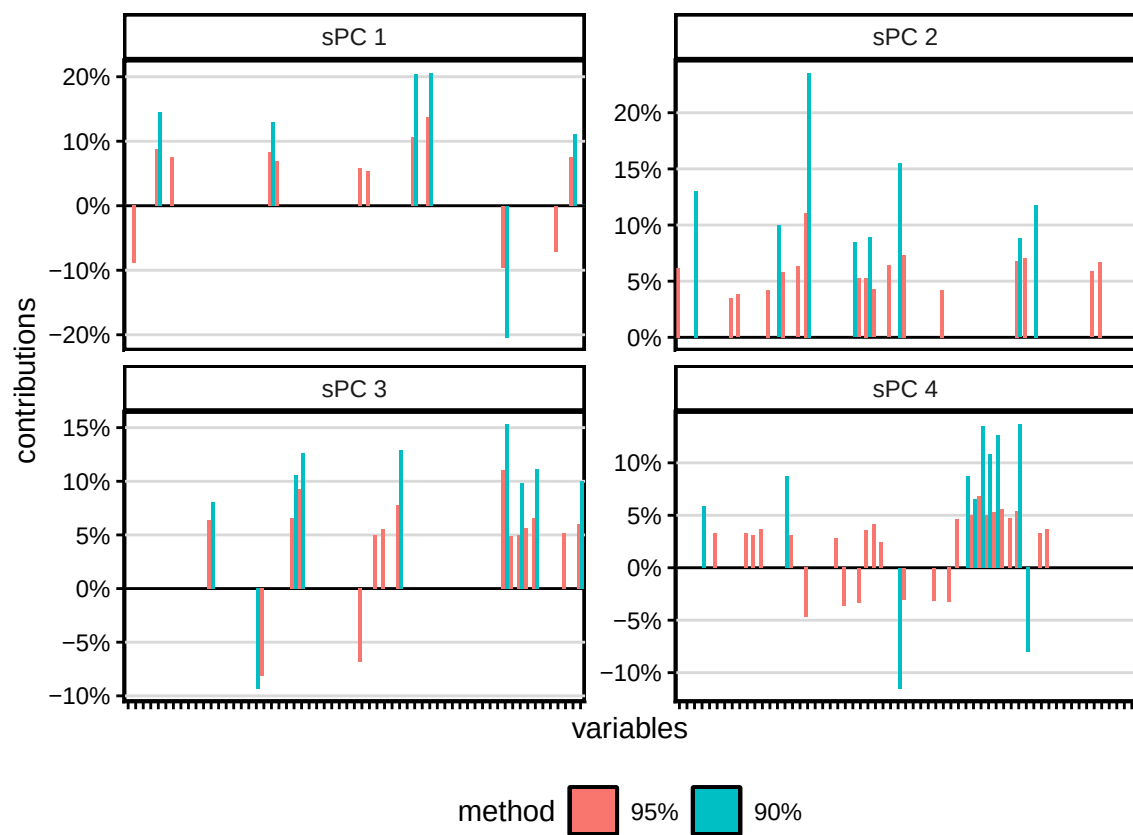


Figure 7: MSSCQ contributions computed with alpha equal to 0.95 and 0.90.

```

+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
+   plot_weights = FALSE
+ )

```

Summary statistics

	C1.95%	C1.90%	C2.95%	C2.90%	C3.95%	C3.90%	C4.95%	C4.90%
Vexp	18.4%	18.0%	8.4%	7.5%	5.2%	4.9%	4.3%	4.0%
Cvexp	18.4%	18.0%	26.8%	25.4%	31.9%	30.3%	36.2%	34.3%
Rvexp	95.4%	93.1%	94.6%	84.0%	94.8%	90.1%	94.5%	89.2%
Rcvexp	95.4%	93.1%	95.1%	90.3%	95.1%	90.2%	95.0%	90.1%
Card	5	3	15	5	13	6	22	9
abs_r	0.97	0.96	0.97	0.90	0.97	0.91	0.97	0.89

At $\alpha = 0.95$, the four sPCs have cardinalities 5, 15, 13, and 22, with RCVEXP of at least 95%. At $\alpha = 0.90$, their cardinalities fall to 3, 5, 6, and 9 while RCVEXP remains above 90%.

```

R> compare_sPCA(
+   list(is_cspca95, is_cspca90),
+   x_axis_var_names = FALSE,
+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
+   print_tables = FALSE
+ )

```

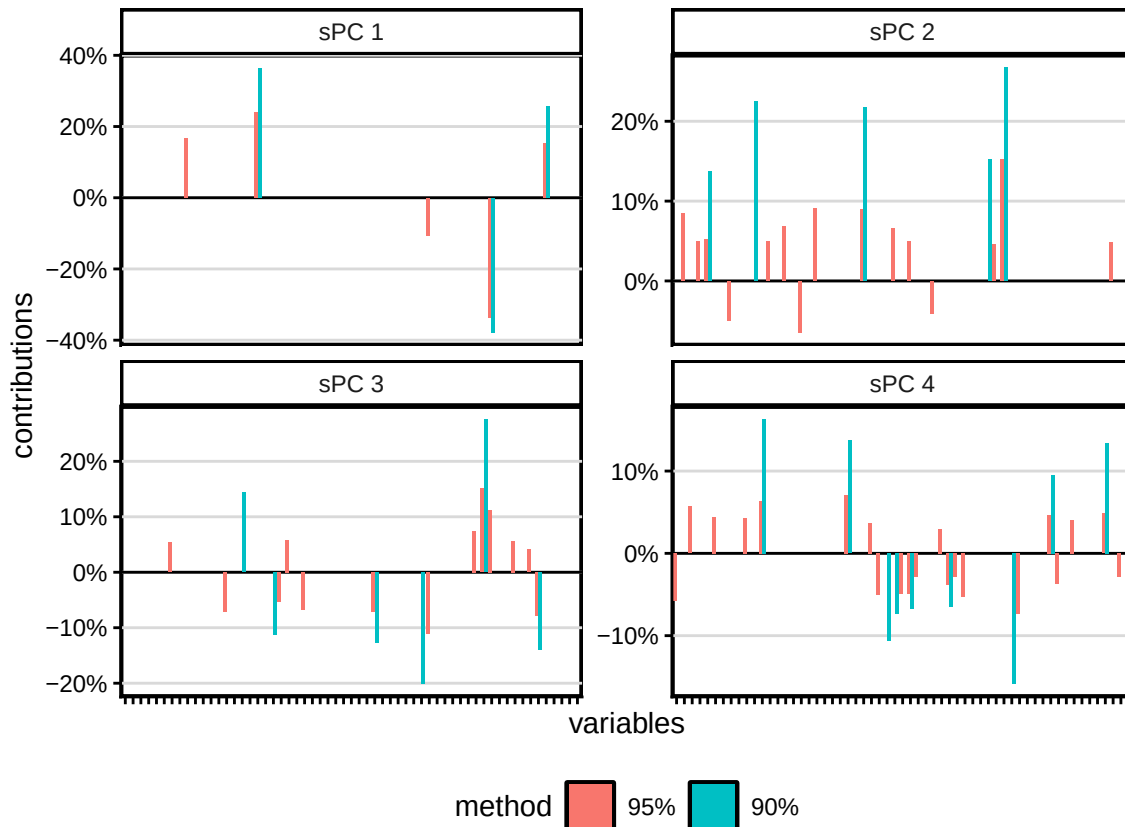


Figure 8: Isolet contributions computed with α equal to 0.95 and 0.90.

These sPCs are also virtually uncorrelated.

```
R> is_cors
```

	95%sPC1	95%sPC2	95%sPC3	95%sPC4	90%sPC1	90%sPC2	90%sPC3	90%sPC4
sPC1	1.00	-0.02	-0.01	-0.01	1.00	-0.02	0.00	-0.01
sPC2	-0.02	1.00	0.00	0.00	-0.02	1.00	-0.03	-0.03
sPC3	-0.01	0.00	1.00	-0.01	0.00	-0.03	1.00	-0.08
sPC4	-0.01	0.00	-0.01	1.00	-0.01	-0.03	-0.08	1.00

Colon data

The `colon` dataset contains expression measurements for 2,000 genes from colon tissue samples collected from 62 patients, 40 of whom had tumours. It is available in the **plsgenomics** package (Boulesteix et al. 2015).

We computed four cSPCA components with `alpha = 0.95` and `alpha = 0.90`.

```
R> cos_cspca95 = spca(cos, n_comps = 4)
R> cos_cspca90 = spca(cos, n_comps = 4, alpha = 0.90)
```

Summary statistics for the Colon fits with `alpha = 0.95` and `alpha = 0.90`.

```
R> compare_spca(
+   list(cos_cspca95, cos_cspca90),
+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
+   plot_weights = FALSE
+ )
```

Summary statistics

	C1.95%	C1.90%	C2.95%	C2.90%	C3.95%	C3.90%	C4.95%	C4.90%
Vexp	42.8%	41.8%	9.4%	8.2%	6.5%	5.9%	5.2%	4.6%
Cvexp	42.8%	41.8%	52.1%	50.1%	58.6%	56.0%	63.9%	60.6%
Rvexp	95.1%	93.1%	95.4%	83.8%	95.9%	87.2%	92.5%	81.6%
Rcvexp	95.1%	93.1%	95.2%	91.4%	95.2%	90.9%	95.0%	90.1%
Card	6	3	12	3	16	2	19	5
abs_r	0.97	0.96	0.97	0.89	0.97	0.80	0.95	0.75

LS-SPCA substantially reduces cardinality in both cases. With `alpha = 0.95`, the four sPCs have 6, 12, 16, and 19 nonzero weights while maintaining RCVEXP of at least 95%. Lowering `alpha` to 0.90 reduces the cardinalities to 3, 3, 2, and 5 while maintaining RCVEXP above 90%. Correlations with the corresponding PCs are at least 0.95 for the first fit and fall to lower values for the second, so the reduction in cardinality is accompanied by weaker agreement with PCA.

For a fat matrix, the rank of the centred data cannot exceed the number of observations minus one: 61 here. Cardinality reductions should therefore be judged against this rank, the largest cardinality that is not redundant. Any component whose support exceeds the rank, including an ordinary PC, can be expressed equally well by any set of variables whose columns form a basis for the same column space. Sparse components for such matrices are consequently subject to an inherent indeterminacy.

The plot shows similar dominant contributions for the first sPC but larger changes in later components. The variables selected at 90% are subsets of those selected at 95% for the first two sPCs; the third pair shares no selected variables and the fourth shares only one.

```
R> compare_spca(
+   list(cos_cspca95, cos_cspca90),
+   x_axis_var_names = FALSE,
+   methods_names = c("95%", "90%"),
+   col_short_names = FALSE,
```

```
+ print_tables = FALSE
+ )
```

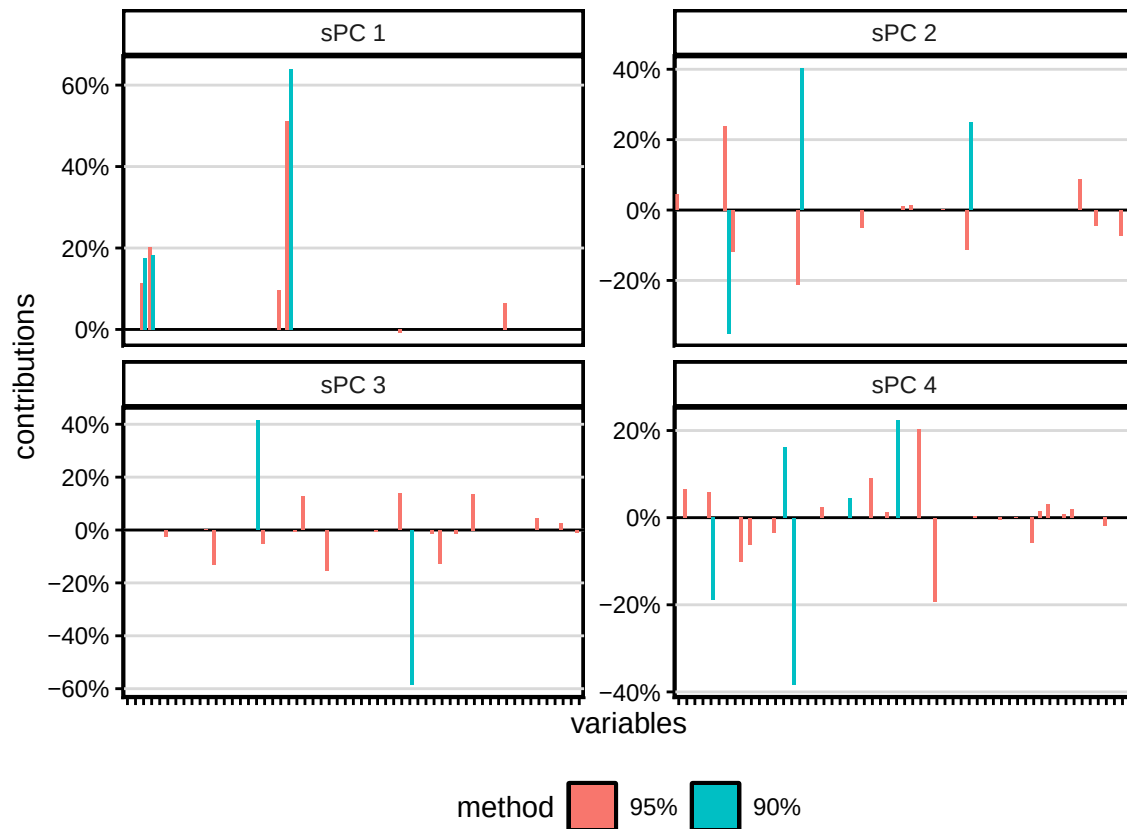


Figure 9: Colon-data contributions computed with alpha equal to 0.95 and 0.90.

The resulting sPCs are also virtually uncorrelated.

```
R> cos_cors
```

	95%SPC1	95%SPC2	95%SPC3	95%SPC4	90%SPC1	90%SPC2	90%SPC3	90%SPC4
sPC1	1.00	0.01	0.01	0.01	1.00	-0.11	-0.02	0.06
sPC2	0.01	1.00	-0.01	-0.02	-0.11	1.00	-0.01	-0.10
sPC3	0.01	-0.01	1.00	0.01	-0.02	-0.01	1.00	0.02
sPC4	0.01	-0.02	0.01	1.00	0.06	-0.10	0.02	1.00

Comparison of LS-SPCA variants

Here we compare LS-SPCA solutions across computational methods, variable selection methods, and values of `alpha`, changing one argument at a time from the defaults.

The data are the Multidimensional Sexual Self-Concept Questionnaire dataset (MSSCQ): after dropping observations with more than three zero values, 16,985 responses on 100 items, centred and scaled to unit variance. We call it `mss`.

Computation methods

Setting `method` to `uspca`, `cspca`, or `pspca` gives the three solutions below.

Comparative summaries for different computation methods.

	method	RCVEXP	Card	Cor.with.PCs	max cor
1	uspca	95.0%	[12, 17, 15, 25]	[0.97, 0.97, 0.96, 0.95]	0.000
2	cspca	95.0%	[12, 17, 15, 25]	[0.97, 0.97, 0.96, 0.95]	0.012
3	pspca	95.0%	[12, 17, 16, 26]	[0.97, 0.97, 0.96, 0.95]	0.013

The three solutions differ little here; as required, the uSPCA sPCs are exactly uncorrelated.

Variable selection methods

This varies the search direction (`var_selection`) and stopping rule (`objective`). Searches use partial squared correlation ("`r2`") to pick candidates, except when `intensive` = `TRUE`, which uses CVEXP. The seven approaches are summarised below.

Comparative summaries for different variable selection methods.

	var sel	RCVEXP	Card	Cor.with.PCs	max cor
1	fwd r2	95.6%	[12, 19, 19, 33]	[0.97, 0.97, 0.97, 0.96]	
2	step r2	95.6%	[12, 19, 19, 32]	[0.97, 0.97, 0.97, 0.96]	
3	bwd r2	95.8%	[13, 19, 20, 32]	[0.98, 0.98, 0.97, 0.95]	
4	fwd cvexp	95.0%	[12, 17, 15, 25]	[0.97, 0.97, 0.96, 0.95]	
5	step cvexp	95.0%	[12, 17, 15, 25]	[0.97, 0.97, 0.96, 0.95]	
6	bwd cvexp	95.0%	[12, 17, 15, 21]	[0.97, 0.97, 0.96, 0.85]	
7	intensive	95.0%	[12, 16, 14, 18]	[0.97, 0.97, 0.95, 0.8]	
					max cor
1					0.015
2					0.015
3					0.019
4					0.012
5					0.012
6					0.034
7					0.009

The `r2` objective gives slightly higher cardinality and sPCs marginally more correlated with the PCs; the `intensive` search is the most parsimonious. Under `r2` the fourth sPC has notably higher cardinality than under `cvexp`.

alpha

`alpha` sets the target minimum CVEXP (or correlation with the residual PCs). Lower `alpha` moves sPCs further from the PCs but lowers cardinality. The summaries below use `alpha` = `c(0.90, 0.95, 0.98)`.

Comparative summaries of solutions obtained with increasing values of alpha.

	alpha	RCVEXP	Card	Cor.with.PCs	max cor
1	0.90	90.1%	[6, 8, 9, 10]	[0.95, 0.93, 0.93, 0.68]	0.080
2	0.95	95.0%	[12, 17, 15, 25]	[0.97, 0.97, 0.96, 0.95]	0.012
3	0.98	98.0%	[25, 30, 29, 40]	[0.99, 0.99, 0.99, 0.98]	0.010

As expected, `alpha` = 0.98 gives parsimonious versions of the PCs.

Practical recommendations

The default settings provide a practical starting point for most analyses. Forward selection is recommended for routine use; stepwise and intensive selection are more suitable for smaller problems, while backward elimination can be expensive. The power method is mainly useful for large matrices.

Setting	Small	Medium	Large
Stepwise selection	Consider	Consider	Avoid
Intensive selection	Consider	Use cautiously	Avoid
Backward elimination	Use cautiously	Avoid	Avoid
objective = "cvexp"	Consider	Consider	Use cautiously
Power method	Little expected gain	Little expected gain	Consider

Matrix size refers mainly to the number of variables for the tall-matrix engine and to both dimensions for the fat-matrix engine. The recommendations concern computational cost rather than guaranteed improvements in the solution.

Comparison with conventional SPCA

We contrast LS-SPCA with conventional SPCA, computed by *elasticnet* 4.1-8 (Zou et al. 2006; Zou and Hastie 2020), the foundational and most-cited implementation. Conventional methods optimise the variance of the sPCs rather than the data approximation; for broader comparisons see Camacho et al. and references therein. We label the solutions **ls-sPCA** and **en-sPCA**.

Since *elasticnet* requires penalty tuning, we fixed the number of components and matched the cardinalities returned by the default `spca()`, setting `sparse = "varnum"` and leaving other arguments at their defaults. This isolates the difference between the two objectives.

Tall matrices

On the MSSCQ data, four sPCs were computed with cardinalities 13, 20, 21, and 30 from the default `spca()`. The two fits are compared below.

Comparative summaries for default spca() (ls-sPCA) and elasticnet spca() (en-sPCA).

Summary statistics

	C1.l _s	C1.e _l	C2.l _s	C2.e _l	C3.l _s	C3.e _l	C4.l _s	C4.e _l
Vexp	23.2%	20.8%	9.4%	9.3%	6.7%	6.6%	3.2%	5.5%
Cvexp	23.2%	20.8%	32.7%	30.1%	39.4%	36.7%	42.6%	42.2%
Rvexp	95.2%	85.0%	94.7%	93.6%	94.9%	93.5%	94.9%	163.2%
Rcvexp	95.2%	85.0%	95.0%	87.5%	95.0%	88.5%	95.0%	94.2%
Card	12	12	17	17	15	15	25	25
abs_r	0.97	0.91	0.97	0.75	0.96	0.79	0.95	0.09

The **ls-sPCA** solution keeps CVEXP above 95% throughout and consistently above **en-sPCA**, though the gap narrows with more components. The en-sPCs correlate far less with their PCs, especially at higher order, and are much more mutually correlated, whereas the ls-sPCs are nearly uncorrelated.

Because conventional SPCA favours correlated variables while LS-SPCA favours less correlated ones, the en-sPCs load more within single scales. The contributions aggregated by scale follow.

Contributions aggregated by scale.

	ls-sPC1	ls-sPC2	ls-sPC3	ls-sPC4	en-sPC1	en-sPC2	en-sPC3	en-sPC4
SAN	-8.8%	6.2%						31.0%
SSE	8.8%			3.3%	32.5%			
SC	7.5%	7.3%				4.7%		
MTA			6.4%	10.0%				
CLS		10.0%		3.1%				2.1%

SP		17.4%	-8.2%	-4.7%		33.6%	
SAS	8.3%					1.3%	-3.6%
SO	6.8%			2.8%			-7.1%
SPS		5.2%	15.8%	-7.0%		24.0%	
SMN		9.6%		10.2%			2.9%
SMT	5.9%	13.7%	-6.9%	-3.0%	45.5%		
SPM	5.3%	4.2%	18.4%	-6.4%		38.9%	
SE	10.7%			4.6%	36.1%		-9.4%
SS	13.7%			11.9%	31.4%		-2.5%
POS		6.8%		26.0%			4.9%
SSS		7.0%			14.0%		
FOS	-9.6%		11.1%	7.0%	-0.9%		17.2%
SPP			22.1%			25.3%	
SD	-7.1%	12.6%					19.2%
ISC	7.5%		11.2%			11.8%	

As expected, the conventional solutions load on fewer scales than LS-SPCA.

Fat matrices

For fat matrices ($n < p$), conventional SPCA can return weights with cardinality above the rank ($n - 1$), even though any combination of the variables can be written with at most ($n - 1$) of them (e.g., Merola and Chen 2019). We illustrate with the **gasoline** dataset (402 near-infrared readings on 60 samples) from *pls* (Liland et al. 2026). The table below compares an `ls-sPCA` fit (`alpha = 0.999`) with *elasticnet* and *abess* `abessPCA()` solutions, both fixed at cardinality 100.

Summary statistics comparing the cspca alpha = 0.999 solution with two conventional SPCA solutions, computed with elasticnet spca() (en) and abess abessPCA() (ab), both fixed at cardinality 100.

Summary statistics

	C1.ls	C1.en	C1.ab	C2.ls	C2.en	C2.ab
Vexp	71.5%	71.6%	69.0%	16.8%	16.8%	15.6%
Cvexp	71.5%	71.6%	69.0%	88.3%	88.4%	84.7%
Rvexp	99.9%	100.0%	96.4%	99.9%	100.0%	93.0%
Rcvexp	99.9%	100.0%	96.4%	99.9%	100.0%	95.8%
Card	16	100	100	16	100	100
abs_r	1.00	1.00	0.98	1.00	1.00	0.51

The maximum meaningful cardinality is 59, yet both **en-sPCA** and **ab-sPCA** return weights of cardinality 100, while **ls-sPCA** reaches 0.999 CVEXP with cardinalities of 6 and 8. Both **ls-sPCA** and **en-sPCA** recover the first two PCs almost exactly (RCVEXP 100%, correlation 1). The **ab-sPCA** solution is less accurate (second sPC at 95.8% RCVEXP, correlation 0.51) and its two sPCs remain correlated at 0.84, whereas the **ls-sPCA** and **en-sPCA** components are effectively uncorrelated.

References

- Camacho J, Smilde AK, Saccenti E, Westerhuis JA (2020). “All sparse PCA models are wrong, but some are useful. Part I: Computation of scores, residuals and explained variance.” *Chemometrics and Intelligent Laboratory Systems*, **196**, 103907. doi:10.1016/j.chemolab.2019.103907.
- Ferrara C., Martella F, Vichi M (2019). “Probabilistic Disjoint Principal Component Analysis.” *Multivariate Behavioral Research*, **54**(1), 47–61.
- Merola G.M. (2015). “Least Squares Sparse Principal Component Analysis: a Backward Elimination approach to attain large loadings.” *Australia & New Zealand Journal of Statistics*, **57**, 391–429. doi:10.1111/anzs.12128.

- Merola G.M., Chen G (2019). “Projection sparse principal component analysis: An efficient least squares method.” *Journal of Multivariate Analysis*, **173**, 366–382. doi:10.1016/j.jmva.2019.04.001.
- Boulesteix A-L., Durif G., Lambert-Lacroix S., Peyre J., Strimmer K. (2024). *plsgenomics: PLS Analyses for Genomics*. R package version 1.5-3. <https://CRAN.R-project.org/package=plsgenomics>.
- Cole R., Fanty M. (1991). *ISOLET*. UCI Machine Learning Repository. doi:10.24432/C51G69. <https://archive.ics.uci.edu/dataset/54/isolet>.
- Open-Source Psychometrics Project (2019). *Raw data from online personality tests*. Updated 5 November 2019. <http://openpsychometrics.org/tests/MSSCQ.php>.
- Revelle W. (2025). *psychTools: Tools to Accompany the ‘psych’ Package for Psychological Research*. R package version 2.5.7. Northwestern University, Evanston, Illinois. <https://CRAN.R-project.org/package=psychTools>.
- Wachter K.W. (1976). “Probability plotting points for principal components.” In *Ninth Interface Symposium Computer Science and Statistics*, 299–308. Prindle, Weber and Schmidt, Boston.
- Winkler A.M. (2021). “How many principal components?” *Brainder*, blog post, 5 April 2021. <https://brainder.org/tag/wachter-test/>.
- Zou H., Hastie T., Tibshirani R. (2006). “Sparse Principal Component Analysis.” *Journal of Computational and Graphical Statistics*, **15**(2), 265–286.
- Zou H, Hastie T (2020). *elasticnet: Elastic-Net for Sparse Estimation and Sparse PCA*. R package version 1.3. <https://CRAN.R-project.org/package=elasticnet>.
- Liland K.H., Mevik B-H., Wehrens R. (2026). *pls: Partial Least Squares and Principal Component Regression*. R package version 2.9-0. <https://CRAN.R-project.org/package=pls>.